

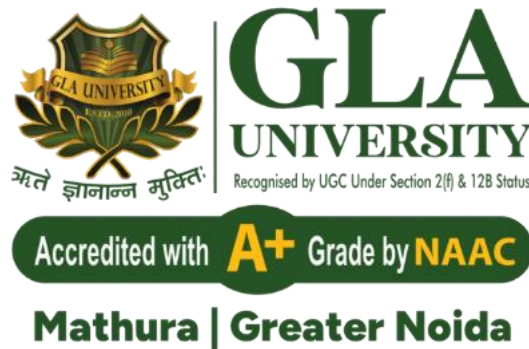
A

SOFTWARE REQUIREMENTS SPECIFICATION

for

GAME SENSE

A Tactical Intelligence Platform for Esports Strategy Planning & Visualization



Prepared By:

Prakarsh Kaushik

Under the Guidance of:

Mr. Gautam Mukherjee

(Lecturer)

Department of Computer Science & Engineering
University Bachelor of Technology, GLA University, Greater Noida

Table of Contents

1 INTRODUCTION

1.1	Purpose	5-5
1.2	Scope of the Product	5-5
1.3	Definitions, Acronyms, and Abbreviations	5-6
1.4	References	6-6
1.5	Overview of the Document	6-6

2 GENERAL DESCRIPTION

2.1	Product Perspective	6-7
2.2	Product Functions	7-7
2.3	User Characteristics	7-7
2.4	Operating Environment	8-8
2.5	Design and Implementation Constraints	8-8
2.6	Assumptions and Dependencies	8-8
2.7	User Documentation	8-8

3 SPECIFIC REQUIREMENTS

3.1	Module-wise Functional Requirements	9-11
3.2	Game & Map Catalog	11-11
3.3	Editor Tool Taxonomy	11-12
3.4	Use Case Descriptions	12-12
3.4.1	Use Case: Select Game	12-12
3.4.2	Use Case: Select Map	12-13
3.4.3	Use Case: Create Strategy on the Editor	13-13
3.4.4	Use Case: Manage Player Units & Vision	13-14
3.4.5	Use Case: Draw Tactical Paths	14-14
3.4.6	Use Case: Add Shapes and Tactical Areas	14-15
3.4.7	Use Case: Annotate with Text, Emojis, and Images	15-15
3.4.8	Use Case: Undo and Redo	15-16
3.4.9	Use Case: Export Strategy as PNG	16-16
3.4.10	Use Case: Get AI Tactical Advice	16-17

4 NON-FUNCTIONAL REQUIREMENTS

4.1	Performance	17-17
4.2	Usability	17-17
4.3	Reliability	17-18
4.4	Portability	18-18
4.5	Security	18-18

4.6	Maintainability	18-18
4.7	Scalability	18-18
5	EXTERNAL INTERFACE REQUIREMENTS	
5.1	User Interfaces	19-19
5.2	Software Interfaces	19-19
5.3	Hardware Interfaces	19-19
5.4	Communications Interfaces	19-20
6	SYSTEM MODELS	
6.1	Use Case Summary	20-20
6.2	Architectural Flow	20-20
6.3	Data Flow Description	20-20
6.4	Diagram Placeholders	21-21
7	SYSTEM REQUIREMENTS	
7.1	Hardware Requirements	21-21
7.2	Software Requirements (Runtime / User)	21-21
7.3	Software Requirements (Development)	21-21
8	ACCEPTANCE TEST CRITERIA	
8.1	Functional and Module Testing	22-22
8.2	End-to-End Workflow Testing	22-22
8.3	Editor and Player Unit Testing	22-22
8.4	Undo/Redo and Export Testing	22-22
8.5	AI Tactical Advisor Testing	22-22
8.6	Cross-Browser and Responsive Testing	22-22
9	FUTURE ENHANCEMENTS	
9.1	Real-Time Collaboration	22-22
9.2	Cloud Storage and Saving	22-22
9.3	Replay Integration	22-22
9.4	Advanced AI Support	22-22
9.5	Custom Map Editor	22-22
9.6	Mobile Support	22-22
9.7	Team Management	22-22
9.8	Enhanced Export Options	22-22

Appendix A: Project Timeline	23-23
Appendix B: Developer Role	23-23
Appendix C: Glossary References (Extended)	23-23

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the complete functional and non-functional requirements for Game Sense, a web-based tactical intelligence platform. The application is designed to assist esports players, coaches, and training communities in planning, visualizing, and refining competitive strategies.

This document is intended to be used as the single baseline reference for the design, development, testing, and evaluation of the application. It is written primarily for the developer, the project guide, and the evaluators, and it establishes a clear agreement between all stakeholders regarding what the system shall and shall not do.

1.2 Scope of the Product

Game Sense is a lightweight, client-side web application that provides an interactive digital strategy board for tactical planning. Users can select a game and a specific map from an internal catalog, place player units with vision fields, draw movement paths, highlight tactical areas, and annotate plays using shapes, text, emojis, and imported images. Completed strategies can be exported as high-quality PNG images for sharing and team briefings.

A key differentiator of the system is its AI-assisted tactical advisor, which analyzes the current board layout and provides contextual coaching suggestions through an integrated chat interface. The system operates entirely on the client side and does not require a dedicated backend server for its core functionality.

The scope explicitly includes: game selection, map selection, interactive strategy editing, drawing and annotation tools, player unit management with vision cones, undo/redo, PNG export, and AI-based tactical assistance. The scope explicitly excludes: live gameplay, match simulation, full multiplayer collaboration, and persistent cloud storage, which are planned as future enhancements.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
UI / UX	User Interface / User Experience
SPA	Single Page Application
SVG	Scalable Vector Graphics - the technology used to render the interactive strategy board
Canvas	The HTML5 canvas element used only at export time to rasterize the strategy to PNG
ViewBox	The 1000 x 562.5 coordinate space in which the strategy board is rendered
Vision Cone	The visual field-of-view wedge rendered from a player unit on the map
Callsign	A short user-defined label attached to a player unit

Data URL	A base64-encoded representation of a locally imported image
Tactical Advisor	The in-application AI assistant that provides coaching suggestions through a chat interface
HashRouter	Client-side routing strategy used by React Router (URLs prefixed with '#/')
React	A JavaScript/TypeScript library for building user interfaces
TypeScript	A strongly typed superset of JavaScript
Vite	A modern build tool and development server
React Router	Client-side routing library used for navigation
Tailwind CSS	Utility-first CSS framework used for the visual design
Gemini API	Google Generative AI API used for AI-assisted coaching
GoogleGenAI SDK	The '@google/genai' client library used to call the Gemini API
npm	Node.js package manager
Node.js	JavaScript runtime used during development and build
Esports	Organized, competitive video gaming
Playbook	A documented set of prepared strategies or plays

1.4 References

- IEEE Std 830-1998 / ISO-IEC-IEEE 29148: Recommended Practice for Software Requirements Specifications.
- Game Sense source code (React + TypeScript + Vite) provided in the project repository.
- React Documentation - <https://react.dev/>
- TypeScript Documentation - <https://www.typescriptlang.org/docs/>
- Vite Documentation - <https://vite.dev/>
- React Router Documentation - <https://reactrouter.com/>
- Tailwind CSS Documentation - <https://tailwindcss.com/>
- Google Generative AI (Gemini) Documentation - <https://ai.google.dev/>

1.5 Overview of the Document

The remainder of this document is organized as follows. Section 2 provides a general description of the product, its perspective, users, operating environment, and design constraints. Section 3 details the specific functional requirements, organized by module, including the game and map catalog, the editor tool taxonomy, and detailed use case descriptions. Section 4 specifies the non-functional requirements. Section 5 describes external interface requirements. Section 6 presents the system models and architectural flow. Section 7 lists the hardware and software requirements for running the application. Section 8 defines the acceptance test criteria, and Section 9 covers future enhancements. Appendices are provided at the end for the project timeline and developer roles.

2. General Description

2.1 Product Perspective

Game Sense is a standalone front-end web application built with React and TypeScript. The application is architected as a lightweight, client-side system: all game and map catalog data is bundled with the application, the strategy editor renders on an SVG board entirely in the

browser, and the optional AI tactical advisor calls the Google Gemini API directly from the client. Client-side routing is implemented using React Router in HashRouter mode. This architecture keeps deployment simple and makes the application fast and easy to share.

The system supplements traditional methods of strategy preparation such as handwritten notes, screenshots, and static diagrams by providing a structured, interactive, and game-specific planning environment.

2.2 Product Functions

The following high-level functions are provided by the system:

- **Game selection:** Select a game from the internal catalog of supported esports titles.
- **Map selection:** Select a map for the chosen game, with search-by-name and game-mode filtering.
- **Strategy editing:** Create and edit tactical plans on an interactive SVG strategy board.
- **Player units & vision:** Manage player units, each with a callsign, color, deployment scale, rotation, and a configurable vision cone.
- **Drawing tools:** Draw freehand pens, arrows, dotted paths, rectangles, and circles.
- **Annotations:** Add text notes, emojis, and locally imported image overlays.
- **Undo / redo:** Undo and redo editing operations (capped at the last 50 states).
- **Board controls:** Zoom (with reset), pan, fullscreen mode, and board lock.
- **PNG export:** Export the completed strategy board as a high-quality PNG image.
- **AI assistance:** Chat with the AI tactical advisor, which analyzes the live board state.

2.3 User Characteristics

The application targets the following user classes. The primary users are esports players and coaches; content creators and training communities form the secondary user base.

User Class	Expected Level of Skill	How the Users Use the System
Esports Players	Basic computer and browser skills; familiar with game maps and terminology	Create and iterate on individual strategies; study maps, routes, and vision setups.
Coaches / Team Strategists	Good tactical knowledge; comfortable with annotation tools	Design team playbooks, assign player roles, draw movement paths, and brief the team.
Content Creators	Basic editing experience	Prepare visual strategy content for videos, streams, and tutorials, and export as PNG.
Training Communities / Academies	Basic browser skills	Host tactical discussions and training sessions using the digital strategy board.

2.4 Operating Environment

- Google Chrome, Microsoft Edge, Mozilla Firefox, or any modern browser with full SVG rendering support.
- No server-side infrastructure is required for the core features; the application runs fully in the browser.

- An internet connection is required only for the optional AI tactical advisor feature and for installing/building the application.
- A Gemini API key must be configured through the environment for the AI feature (no key is committed to the repository).
- During development, Node.js and npm are required to run the Vite development server and build the production bundle.

2.5 Design and Implementation Constraints

- The application must be implemented as a client-side web application with no backend server.
- The technology stack is fixed to React, TypeScript, and Vite for this phase.
- The strategy board must be rendered using SVG in a 1000 x 562.5 viewBox coordinate space.
- Game and map catalog data are bundled locally in a constants module; no dynamic content from a server is expected.
- The user interface follows a dark, esports-themed visual style with a rose/red accent color (#FF2D55) consistent across all modules.
- The codebase must be modular and component-based to support future enhancements.
- Core editing functionality must remain usable even if the AI service is unreachable.

2.6 Assumptions and Dependencies

- Users access the application through a modern browser with SVG and canvas support.
- A stable internet connection is available when the AI tactical advisor is used.
- The Google Gemini API free tier remains available for the duration of the project.
- The developer has a working knowledge of React, TypeScript, and modern web tooling.
- Supported games and maps are predefined and bundled; no external game data sources are required in this phase.

2.7 User Documentation

- In-application tooltips for every editor tool and board control.
- This SRS document, which serves as the requirements baseline.
- A project README describing installation, build, and deployment steps.

3. Specific Requirements (Functional Requirements)

This section defines the functional requirements of Game Sense. Requirements are organized module-wise and are uniquely identified by the FR-GS-xx prefix. The priority of each requirement is marked as High, Medium, or Low.

3.1 Module-wise Functional Requirements

Req. ID	Module	Functional Requirement	Priority
FR-GS-01	Home	The system shall display a home page with a hero section, rotating atmospheric background imagery, and a clear call-to-action to begin the strategy planning workflow.	High
FR-GS-02	Home	The header and footer shall provide navigation to the Game Selection page and a link to the project's community.	Medium
FR-GS-03	Game Selection	The system shall display a catalog of predefined games (Call of Duty Mobile, Valorant, Counter-Strike 2, League of Legends) with name, category, and description.	High
FR-GS-04	Game Selection	On selection of a game, the system shall navigate the user to the Map Selection module for the chosen game.	High
FR-GS-05	Map Selection	The system shall display the maps associated with the selected game, including thumbnail and game-mode badges.	High
FR-GS-06	Map Selection	The system shall provide a search-by-name box and a mode filter (ALL, HP, S&D, CTL, COMPETITIVE, RANKED) for games that define modes.	Medium
FR-GS-07	Map Selection	On selection of a map, the system shall navigate the user to the Strategy Editor with the chosen map loaded on the board.	High
FR-GS-08	Map Selection	If no maps match the current search/filter, the system shall display an empty state with a reset option.	Medium
FR-GS-09	Strategy Editor	The editor shall render the selected map background on an SVG board within a 1000 x 562.5 viewBox coordinate space.	High
FR-GS-10	Strategy Editor	The editor shall initialize two default player units (one per side) with vision cones on load; these units shall be permanent and not deletable.	High
FR-GS-11	Strategy Editor	The editor shall support zooming with a live percentage indicator and a reset-zoom action.	Medium
FR-GS-12	Strategy Editor	The editor shall support panning of the board and a fullscreen toggle.	Medium
FR-GS-13	Strategy Editor	A Reset action shall clear all tactical	Medium

		elements and restore the default board after explicit user confirmation.	
FR-GS-14	Strategy Editor	The tool dock shall provide the following tools: Select, Pen, Arrow, Dotted path, Rectangle, Circle, Text, Emoji, Image, Eraser/Clear, and Lock.	High
FR-GS-15	Strategy Editor	The Select tool shall allow elements to be selected, dragged, resized, rotated, recolored, and deleted.	High
FR-GS-16	Strategy Editor	Pen, Arrow, and Dotted-path tools shall render strokes with configurable color and size.	High
FR-GS-17	Strategy Editor	Rectangle and Circle tools shall render filled/bordered shapes with configurable color and size.	High
FR-GS-18	Strategy Editor	The Text tool shall place editable labels; double-clicking a text element shall allow in-place editing.	High
FR-GS-19	Strategy Editor	The Emoji tool shall place emoji symbols with configurable size, rotation, and opacity.	Medium
FR-GS-20	Strategy Editor	The Image tool shall import an image from the local device (as a data URL) and place it on the board.	Medium
FR-GS-21	Strategy Editor	The Eraser/Clear tool shall remove placed elements, and the Lock tool shall toggle editing of the whole board.	Medium
FR-GS-22	Player Units	Player units shall expose a callsign label and configurable color, deployment scale, rotation, vision field (10-120 degrees), engagement depth (20-500 m), and show/hide of the vision cone.	High
FR-GS-23	Properties Panel	The settings sidebar shall expose per-element properties, including a color system with presets and a custom color picker.	Medium
FR-GS-24	Undo / Redo	The system shall support undo and redo via toolbar buttons and keyboard shortcuts (Ctrl+Z, Ctrl+Y, Ctrl+Shift+Z), with history capped at the last 50 states.	High
FR-GS-25	AI Assistance	The AI tactical advisor shall summarize the live board state (player units with position, rotation, vision; drawings; notes) and send it together with the conversation history to the Gemini API (gemini-3-flash-preview).	High
FR-GS-26	AI Assistance	AI responses shall be rendered with markdown formatting (bold, headings, bullet points).	Medium
FR-GS-27	AI Assistance	If the AI service fails or is unreachable, the system shall display a graceful error message and the editor shall remain fully functional.	High

FR-GS-28	Export	The system shall export the board (map background plus all tactical elements) as a high-quality PNG image named "{game}-{map}-strategy-{timestamp}.png".	High
FR-GS-29	Navigation	The editor HUD shall show breadcrumb navigation back to the Games and Map Selection pages.	Medium
FR-GS-30	Session	On a fresh session (first visit to the tab), the system shall redirect the user to the Home page.	Medium
FR-GS-31	Responsiveness	The system shall provide a responsive layout that renders correctly on desktop, tablet, and mobile screen sizes.	Medium

3.2 Game & Map Catalog

The bundled catalog currently supports four games. Each map is associated with a game and is tagged with one or more game modes used for filtering on the Map Selection page.

Game	Category	Maps (Count)	Modes
Call of Duty Mobile (CODM)	FPS	Summit, Hacienda, Arsenal, Combine, Takeoff, Tunisia, Firing Range, Coastal, Slums, Meltdown, Raid, Standoff, Crossroads Strike (13)	HP, SND, CTL
Valorant	Tactical Shooter	Abyss, Ascent, Bind, Breeze, Corrode, Fracture, Haven, Icebox, Lotus, Pearl, Split, Sunset (12)	Competitive
Counter-Strike 2 (CS2)	FPS	Ancient, Anubis, Dust II, Inferno, Mirage, Nuke, Overpass (7)	Competitive
League of Legends (LoL)	MOBA	Summoner's Rift (1)	Ranked

Additional games and maps can be added by extending the bundled constants module without modifying the core editor, in line with the scalability requirement (NFR-24).

3.3 Editor Tool Taxonomy

Tool	Icon / Key	Purpose
Select	Mouse pointer	Select, drag, resize, rotate, recolor, and delete elements.
Pen	Pencil	Freehand drawing of tactical strokes.
Path (Arrow)	Long arrow	Draw directional movement paths.
Dash (Dotted)	Ellipsis	Draw dashed/dotted movement paths.
Area (Rectangle)	Vector square	Highlight rectangular tactical areas.
POI (Circle)	Circle	Mark points of interest with circular shapes.
Text	Font	Add editable text labels; double-click to edit.
Emoji	Smile	Place emoji symbols with

		size/rotation/opacity controls.
Media (Image)	Image	Import a local image as a board overlay.
Eraser (Clear)	Eraser	Remove placed tactical elements.
Lock	Lock / Unlock	Lock or unlock editing of the whole board.

3.4 Use Case Descriptions

The following use cases describe the detailed interactions between actors (users) and the system. The primary actor for all use cases is the User (esports player, coach, or content creator).

3.4.1 Use Case: Select Game

Field	Details
Use Case No.	UC-01
Use Case Name	Select Game
Actor(s)	User
Purpose	To allow the user to choose a game for which they wish to plan a strategy.
Trigger	User clicks the 'Games' navigation item or the call-to-action on the Home page.
Precondition	Application is loaded and the game catalog is available.
Flow of Events	1. User navigates to the Game Selection page ('/games'). 2. The system displays the catalog of available games with category badges. 2. The user selects a game card. 2. The system stores the selection and navigates to the Map Selection module ('/maps/:gameId').
Alternative Flows	If the catalog is empty, the system displays an empty state; the game list is bundled, so this is not expected in normal operation.
Postconditions	The selected game is used as context for Map Selection; the map page is displayed.
Priority	High
Security	None required.

3.4.2 Use Case: Select Map

Field	Details
Use Case No.	UC-02
Use Case Name	Select Map
Actor(s)	User
Purpose	To allow the user to choose the specific map on which the strategy will be built.
Trigger	User reaches the Map Selection page after selecting a game.
Precondition	A game has been selected (UC-01).
Flow of Events	1. The system displays maps associated with the selected game, with thumbnail and mode badges. 2. The user may search by map name or filter by mode (HP, SND, CTL, COMPETITIVE, RANKED). 2. The user selects a map. 2. The system navigates to the Strategy Editor

	(/editor/:gameId/:mapId').
Alternative Flows	If the search/filter returns no results, the system displays a 'no maps matching current intel' state with a reset option.
Postconditions	The selected map is loaded as the board background in the Strategy Editor.
Priority	High
Security	None required.

3.4.3 Use Case: Create Strategy on the Editor

Field	Details
Use Case No.	UC-03
Use Case Name	Create Strategy on the Editor
Actor(s)	User
Purpose	To enable the user to build a tactical plan on the interactive SVG board.
Trigger	User selects a map (UC-02).
Precondition	A map is loaded on the board; two default player units with vision cones are initialized.
Flow of Events	1. The user selects a tool from the tool dock (select, pen, arrow, dotted, rectangle, circle, text, emoji, image, eraser, lock). 2. The user interacts with the board to place or draw the tactical element. 2. The system renders the element in the 1000 x 562.5 viewBox and records the action in the history stack. 2. The user may select, drag, resize, rotate, recolor, or delete any placed element.
Alternative Flows	Incorrect actions can be undone (UC-08). The board can be zoomed, panned, locked, or reset at any time.
Postconditions	The tactical layout is updated and the history stack is maintained (capped at 50 states).
Priority	High
Security	None required.

3.4.4 Use Case: Manage Player Units & Vision

Field	Details
Use Case No.	UC-04
Use Case Name	Manage Player Units & Vision
Actor(s)	User
Purpose	To allow the user to represent player positions and their fields of view on the map.
Trigger	User selects an existing player unit on the board (two units are pre-placed by default).
Precondition	Strategy Editor is open with a loaded map.
Flow of Events	1. User clicks a player unit to select it. 2. The settings sidebar displays the unit intel panel (callsign, color, deployment scale, vision field, engagement depth). 2. User edits the callsign and adjusts color, scale, rotation, FOV (10-120 deg), depth (20-500 m), and vision cone visibility. 2. User may drag the unit to reposition it; the vision cone updates in

	real time.
Alternative Flows	Player units are permanent (isPermanent = true) and cannot be deleted, ensuring the board always has reference units.
Postconditions	Player units and their vision cones are rendered and reflected in the AI map summary.
Priority	High
Security	None required.

3.4.5 Use Case: Draw Tactical Paths

Field	Details
Use Case No.	UC-05
Use Case Name	Draw Tactical Paths
Actor(s)	User
Purpose	To allow the user to visualize movement routes and attack/defense paths.
Trigger	User selects the pen, arrow, or dotted-path tool.
Precondition	Strategy Editor is open.
Flow of Events	1. User selects a drawing tool (pen, arrow, or dotted path). 2. User draws on the board by dragging the pointer. 2. The system renders the stroke with the selected color and size. 2. User may select and delete or modify the drawn path.
Alternative Flows	Arrow endpoints align with the drag gesture; the user can re-draw or undo to correct mistakes.
Postconditions	Paths are rendered as part of the tactical layout and included in the AI summary.
Priority	High
Security	None required.

3.4.6 Use Case: Add Shapes and Tactical Areas

Field	Details
Use Case No.	UC-06
Use Case Name	Add Shapes and Tactical Areas
Actor(s)	User
Purpose	To highlight areas of interest such as sites, zones, and points of interest.
Trigger	User selects the rectangle or circle tool.
Precondition	Strategy Editor is open.
Flow of Events	1. User selects a shape tool (rectangle or circle). 2. User drags on the board to define the shape bounds. 2. The system renders the shape with the selected color, size, and opacity. 2. User may resize, recolor, or delete the shape.
Alternative Flows	The board can be zoomed for precise placement.
Postconditions	Shapes are rendered on the layout.
Priority	High
Security	None required.

3.4.7 Use Case: Annotate with Text, Emojis, and Images

Field	Details
Use Case No.	UC-07
Use Case Name	Annotate with Text, Emojis, and Images
Actor(s)	User
Purpose	To enrich the strategy with notes, symbols, and overlays.
Trigger	User selects the text, emoji, or image tool.
Precondition	Strategy Editor is open.
Flow of Events	1. User selects an annotation tool. 2. For text: user clicks a location, types the label, and may double-click later to edit it in place. 2. For emojis: user chooses a symbol from the emoji panel and places it, adjusting size, rotation, and opacity. 2. For images: user selects an image file from the local device; the file is read as a data URL and placed at the board center. 2. The system renders the annotation; the user may move, resize, or delete it.
Alternative Flows	Image import uses the browser file picker; only supported image formats are accepted.
Postconditions	Annotations appear on the tactical layout and (for text) are included in the AI map summary.
Priority	Medium
Security	Imported images are processed entirely within the browser as data URLs and are not transmitted to any server.

3.4.8 Use Case: Undo and Redo

Field	Details
Use Case No.	UC-08
Use Case Name	Undo and Redo
Actor(s)	User
Purpose	To allow rapid correction and iteration of the strategy layout.
Trigger	User presses the Undo or Redo button, or uses the keyboard shortcut.
Precondition	At least one editing action has been performed.
Flow of Events	1. User presses the Undo action (button or Ctrl+Z). 2. The system reverts the most recent editing operation and restores the previous state. 2. User presses the Redo action (button, Ctrl+Y, or Ctrl+Shift+Z). 2. The system re-applies the reverted operation.
Alternative Flows	When the history stack is empty, the undo action is a no-op. The history is capped at the last 50 states; the oldest states are discarded beyond that.
Postconditions	The layout reflects the undone/redone state; history and redo stacks are updated.
Priority	High
Security	None required.

3.4.9 Use Case: Export Strategy as PNG

Field	Details
Use Case No.	UC-09
Use Case Name	Export Strategy as PNG
Actor(s)	User
Purpose	To allow the user to share or present the completed strategy as an image.
Trigger	User clicks the Export (download) action in the right action dock.
Precondition	A tactical layout exists on the board.
Flow of Events	1. User clicks the Export button. 2. The system serializes the board SVG, loads the map minimap as the background, and rasterizes the composite onto a canvas. 2. The system triggers a browser download of the PNG file named "{game}-{map}-strategy-{timestamp}.png".
Alternative Flows	If the map background fails to load, the system exports the tactical elements without the background.
Postconditions	A PNG file is downloaded to the user's device.
Priority	High
Security	None required; the export is generated entirely on the client.

3.4.10 Use Case: Get AI Tactical Advice

Field	Details
Use Case No.	UC-10
Use Case Name	Get AI Tactical Advice
Actor(s)	User, Google Gemini API
Purpose	To obtain intelligent, contextual recommendations for improving the current tactical setup through a conversational advisor.
Trigger	User opens the Tactical Advisor panel (right dock) and sends a message.
Precondition	A tactical layout exists; an internet connection and a configured Gemini API key are available.
Flow of Events	1. User clicks the Tactical Advisor button to open the chat panel. 2. The system builds a summary of the live board state (player units with position/rotation/vision, recent drawings, notes). 2. The system sends the summary, the conversation history, and the user's message to the Gemini API (gemini-3-flash-preview). 2. The API returns a tactical response, which is rendered with markdown formatting. 2. The response appears in the chat feed; a loading indicator ('Analyzing Sector...') is shown while waiting.
Alternative Flows	If the API is unreachable or returns an error, the system displays a graceful error message ('Error in tactical uplink') and the editor remains fully usable.
Postconditions	The AI response is displayed in the chat feed; the conversation history is retained for the session.
Priority	High
Security	Only non-sensitive board-layout data and conversation text are sent to the AI service; no personal data is transmitted.

4. Non-Functional Requirements

This section specifies the quality attributes that the system must satisfy.

4.1 Performance

ID	Requirement
NFR-01	The application shall load the home page within 3 seconds on a standard broadband connection.
NFR-02	The strategy editor shall provide smooth interaction (no perceptible lag) while drawing, placing elements, and panning the SVG board.
NFR-03	PNG export shall complete within 5 seconds for a typical strategy layout.
NFR-04	The AI advisor response shall be shown within 15 seconds on a typical internet connection.
NFR-05	The application bundle shall be optimized for size to minimize initial load time.

4.2 Usability

ID	Requirement
NFR-06	The interface shall be intuitive, allowing a first-time user to create a basic strategy without external guidance.
NFR-07	All editing tools shall be discoverable through an organized tool dock with tooltips.
NFR-08	The application shall provide consistent navigation and labeling across all modules.
NFR-09	The dark esports-themed visual design (rose accent #FF2D55) shall be consistent across every page.
NFR-10	Keyboard shortcuts shall be provided for frequent actions such as Undo (Ctrl+Z) and Redo (Ctrl+Y / Ctrl+Shift+Z).

4.3 Reliability

ID	Requirement
NFR-11	Undo/redo history shall be maintained consistently without data loss during normal usage; the history is capped at the last 50 states.
NFR-12	Failure of the AI service shall not disrupt the strategy editor or cause loss of user work.
NFR-13	The application shall not crash on unexpected user input, empty states, or missing map assets.
NFR-14	Destructive actions (such as Reset) shall require explicit user confirmation before execution.

4.4 Portability

ID	Requirement
NFR-15	The application shall run identically on the latest versions of Google Chrome, Microsoft Edge, and Mozilla Firefox.
NFR-16	The application shall function on Windows, macOS, and Linux operating systems through the browser.
NFR-17	The interface shall be responsive across desktop, tablet, and mobile screen sizes.

4.5 Security

ID	Requirement
NFR-18	No sensitive personal data shall be collected or stored by the application.
NFR-19	The Gemini API key shall be provided through the environment (process.env) and shall never be committed to the public bundle or repository.
NFR-20	User-imported images shall be handled as in-memory data URLs within the browser and never transmitted to any server.
NFR-21	Only board-layout data required for context shall be sent to the AI service; no personal data shall be transmitted.

4.6 Maintainability

ID	Requirement
NFR-22	The codebase shall follow a modular, component-based structure (pages, services, constants, types) to allow independent updates of each module.
NFR-23	TypeScript shall be used to provide type safety and improve code maintainability.
NFR-24	The source code shall be managed using Git for version control and collaboration.
NFR-25	The project shall be documented with a README covering setup, build, and deployment.

4.7 Scalability

ID	Requirement
NFR-26	New games and maps shall be added by extending the bundled catalog constants without rewriting the core editor.
NFR-27	The modular design shall allow future integration of real-time collaboration, cloud storage, and analytics.

5. External Interface Requirements

5.1 User Interfaces

The application exposes the following primary user interfaces:

- Home Page - hero landing section with rotating backgrounds, header/footer navigation, and a call-to-action.
- Game Selection Page ('/games') - presents the catalog of games (the Arena).
- Map Selection Page ('/maps/:gameId') - presents maps for the selected game with search and mode filters.
- Strategy Editor ('/editor/:gameId/:mapId') - provides the SVG board, the left tool dock, the settings sidebar, the right action dock (AI advisor, reset, export), and the AI chat panel.
- Tactical Advisor Panel - chat interface for the AI tactical assistant.

Visual standards: a dark tactical theme with a rose/red accent color (#FF2D55), consistent typography (Orbitron display font with system fallbacks), tooltips on all tools, and a consistent HUD layout across the editor.

5.2 Software Interfaces

Interface	Description
Google Gemini API (GoogleGenAI SDK)	REST-based generative AI service invoked from the client through '@google/genai'. Model: gemini-3-flash-preview. Communication over HTTPS using JSON request/response; the live board state is summarized and sent as chat context.
SVG Rendering	The strategy board and all tactical elements are rendered as inline SVG within a 1000 x 562.5 viewBox.
HTML5 Canvas (export only)	Used at export time to rasterize the serialized SVG plus map background into a downloadable PNG.
Browser File API / FileReader	Used to import local image overlays as data URLs.
Blob / URL API	Used to create the SVG blob and PNG download link during export.
sessionStorage	Used to track whether a session is active and to redirect fresh sessions to the Home page.
React Router (HashRouter)	Client-side routing between '/', '/games', '/maps/:gameId', and '/editor/:gameId/:mapId'.

5.3 Hardware Interfaces

- Mouse / trackpad for drawing, dragging, and precise editing.
- Keyboard for shortcuts and text input.
- Touchscreen for direct interaction on tablets and mobile devices.
- Display (HD or higher resolution) for accurate map visualization.

5.4 Communications Interfaces

- HTTPS for all network communication (Gemini API calls).
- No server-to-client communication channels are required for core functionality.
- WebSocket may be used in future phases for real-time collaboration (out of scope for this version).

6. System Models

6.1 Use Case Summary

The system is used by a single primary actor, the User, supported by the Google Gemini API as a secondary actor. The summary of use cases is as follows:

Use Case ID	Use Case Name	Actor(s)
UC-01	Select Game	User
UC-02	Select Map	User
UC-03	Create Strategy on the Editor	User

UC-04	Manage Player Units & Vision	User
UC-05	Draw Tactical Paths	User
UC-06	Add Shapes and Tactical Areas	User
UC-07	Annotate with Text, Emojis, and Images	User
UC-08	Undo and Redo	User
UC-09	Export Strategy as PNG	User
UC-10	Get AI Tactical Advice	User, Google Gemini API

6.2 Architectural Flow

The application follows a linear, route-based workflow implemented with React Router in HashRouter mode:

1. The user opens the application and lands on the Home page (fresh sessions are redirected here).
2. The user selects a game from the available list ('/games').
3. The user chooses a map related to that game ('/maps/:gameId').
4. The user enters the Strategy Editor ('/editor/:gameId/:mapId').
5. The user creates and edits tactical plans using the visual tools on the SVG board.
6. The user may invoke the AI tactical advisor to obtain tactical feedback.

Each module is treated as an independent page component. The client-side state (selected game, map, and board elements) is managed within the editor component, and the game/map catalog is served from a bundled constants module.

6.3 Data Flow Description

All game and map catalog data is bundled statically in the application. The strategy layout is represented in-memory as a list of tactical elements (players, lines, arrows, shapes, text, emojis, and images) rendered onto an SVG board. The undo/redo history is maintained as a stack of board states capped at 50 entries. On export, the board SVG is serialized, composited over the map minimap, and rasterized to a PNG on the client. On AI request, the board state is summarized into a text context, combined with the conversation history, and sent to the Gemini API; the response is rendered in the chat panel.

6.4 Diagram Placeholders

The following diagrams shall be added to this section during the design phase: (a) Use Case Diagram, (b) Module Architecture Diagram, (c) Application Flow / Wireflow Diagram, and (d) Data Flow Diagram (DFD Level 0).

7. System Requirements (Hardware & Software)

7.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / equivalent	Intel i5 / AMD Ryzen 5 or higher
RAM	4 GB	8 GB
Storage	500 MB free space	10 GB free space
Display	1366 x 768	Full HD (1920 x 1080)

Input Devices	Keyboard and mouse	Keyboard, mouse, and touchscreen
Internet	Required for AI features	Broadband connection

7.2 Software Requirements (Runtime / User)

Software	Requirement
Operating System	Windows 10 or above, macOS, or Linux
Web Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox (latest versions) with SVG support
Internet Connection	Required for AI assistance and online resources

7.3 Software Requirements (Development)

Tool	Purpose
React (19.x)	Component-based user interface library
TypeScript	Strongly typed superset of JavaScript
Vite	Build tool and development server
React Router (7.x, HashRouter)	Client-side routing and navigation
Tailwind CSS	Utility-first styling and responsive design
Google Gemini API (@google/genai)	AI-assisted tactical advice (model: gemini-3-flash-preview)
Font Awesome	Icon set used in the interface
Node.js + npm	Runtime and package manager for development/build
Visual Studio Code	Code editor and debugging
Git / GitHub	Version control and collaboration

8. Acceptance Test Criteria

The application shall be considered accepted when all of the following criteria are met:

Criterion	Verification Method
All FR-GS-01 to FR-GS-31 requirements are implemented and functional.	Functional testing of each module against the requirement list.
Home, game selection, map selection, and editor navigation work end-to-end.	End-to-end workflow test.
All 11 editor tools operate correctly and elements can be added, edited, and removed.	Manual UI testing in the strategy editor.
Player units and vision cones render and update in real time.	Player unit interaction tests.
Undo/redo operate correctly, including keyboard shortcuts and the 50-state history cap.	History stack tests.
The strategy board exports to a valid, high-quality PNG with the expected filename.	Export test on multiple layouts.
The AI tactical advisor returns contextual suggestions or a graceful error when offline.	AI module test with and without network.
The interface renders correctly in Chrome, Edge, and Firefox.	Cross-browser compatibility test.
The layout is responsive across desktop, tablet, and mobile widths.	Responsive design test.
No critical or major defects remain open.	Bug triage against severity levels.

Testing shall cover unit tests for state/history logic, integration tests for module navigation, and user acceptance testing (UAT) by the project guide and the developer.

9. Future Enhancements

The modular architecture of Game Sense has been designed to accommodate the following future enhancements:

- Real-time collaboration - multiple users editing the same strategy board simultaneously.
- Cloud storage and saving - saving and loading strategies from online accounts or databases.
- Replay integration - analyzing actual match replays to study gameplay situations.
- Advanced AI support - deeper tactical analysis, role suggestions, and strategy optimization.
- Custom map editor - allowing users to create or modify their own maps.
- Mobile support - a dedicated mobile application or enhanced mobile experience.
- Team management - roster management, shared notes, and role assignment tools.
- Enhanced export options - exporting strategies as PDF or shareable links.

Appendix A: Project Timeline

Phase	Activity	Duration
Phase 1	Requirement Analysis & Planning	1 Week
Phase 2	UI Design & Wireframing	1 Week
Phase 3	Core Development (Editor, Tools, Maps)	3 Weeks
Phase 4	AI Integration & Export Module	1-2 Weeks
Phase 5	Testing & Bug Fixing	1 Week
Phase 6	Documentation & Final Submission	1 Week

Estimated Total Duration: 8-10 Weeks.

Appendix B: Developer Role

Member	Role	Key Responsibilities
Prakarsh Kaushik	Sole Developer	End-to-end development of Game Sense: system design and architecture; UI/UX design; React and TypeScript implementation; drawing tools, player units, and map visualization; AI tactical advisor integration; PNG export; state and undo/redo history logic; testing, bug fixing, and deployment; documentation and version control.

Note: Game Sense is developed as a solo project by Prakarsh Kaushik.

Appendix C: Glossary References (Extended)

Term	Description
Tactical Layout	The complete set of player units, paths, shapes, and annotations placed on a map.
Playbook	A collection of prepared strategies used by a team.
Zone / Area	A highlighted region on the map used to denote an area of tactical interest.
Player Unit	A symbol representing a player position, with callsign, color, scale, rotation, and vision cone.
Vision Cone	The field-of-view wedge rendered from a player unit.
Session State	The in-memory data retained while the user works in a browser tab.
Board	The SVG strategy surface with a 1000 x 562.5 coordinate space.